

MORAA GemVision

Work Audit Log · Phase 5 Scale Architecture · generated 2026-10-02

How to use this page: tick the box ☐ at the start of a task once you have checked that it is done and done well. Write the status (done, in progress, blocked, not started) by hand in the empty box at the right of each task, and fill in the summary table below. Nothing on this page is pre-filled with a status. The reasons, decisions (D1, D2...) and errors (E1, E2...) are explained once for the whole phase, after the checklist; tasks refer to them by number.

Domain	Tasks	Done	In progress	Blocked	Not started
1. Durable work	4				
2. Providers and fairness	2				
3. Customer experience and edges	2				
4. Not built (owner decisions)	2				
5. Deployment files	1				
Total	11				

Why this phase

A paid order started only as an in-memory background job, so every deploy or crash stranded paid orders until a later sweep refunded them. Every periodic job ran in every server process, a provider outage kept burning calls, nothing capped how much one customer could queue, the processing message always promised 20 to 30 seconds, and there was no way to run the server except by hand. A shop that grows past one phone and one laptop needs these fixed before it needs more customers.

WHAT WAS DONE, IN SHORT

Paid orders, invoices and ops messages are written to a database 'outbox' first and picked up again after a restart; periodic jobs have a one-row lease so only one process runs each; providers have a circuit breaker, a rate smoother and a priority gate; a customer can have at most three paid orders in progress; the processing message uses measured times; middleware was made cheaper; uploads are size-checked as they stream; and Docker files describe how to run the stack. Everything is behind settings and needs no new service such as Redis.

Checklist

1. Durable work

Nothing paid is lost to a restart.

- ☐ **1.1 A paid order is recorded in the database before anything else can fail, and started again if the server dies first** ☐
- FILES** backend/app/services/outbox.py, backend/app/models/outbox_job.py, backend/alembic/versions/0016_outbox_jobs.py, backend/app/api/routes/meta_webhook.py
- HOW TO CHECK** On a test copy: place a test order and kill the server within 5 seconds, restart: the order is delivered (not refunded) within about two minutes. Automated tests cover it; no live run.

- ☐ **1.2 Invoices and ops-team messages are retried with growing waits and parked as dead with an alert after six failures**

FILES backend/app/services/outbox.py, backend/app/services/alert_service.py

HOW TO CHECK Tests in tests/test_outbox.py. In production /metrics shows moraa_outbox_jobs.
- ☐ **1.3 Only one server process runs each periodic job (payment checks, refunds of stuck orders, alerts)**

FILES backend/app/services/scheduler_lease.py, backend/alembic/versions/0017_scheduler_leases.py

HOW TO CHECK Tests in tests/test_scheduler_lease.py. Not exercised with two real processes.
- ☐ **1.4 A photo cut off mid-processing is closed and the customer asked to resend (nothing was charged)**

FILES backend/app/services/meta_whatsapp_service.py

HOW TO CHECK Tests in tests/test_money_recovery.py.

2. Providers and fairness

- ☐ **2.1 A failing provider is skipped for a minute (circuit breaker) and calls can be smoothed to the provider's quota**

FILES backend/app/ai/provider_protection.py, backend/app/ai/image_generation_manager.py

HOW TO CHECK Tests in tests/test_provider_protection.py. IMAGE_PROVIDER_RPM stays 0 (off) until you tell us the quota.
- ☐ **2.2 A single Studio Shot is served ahead of Pack images, and a customer can have at most 3 paid orders in progress**

FILES backend/app/ai/concurrency_gate.py, backend/app/api/routes/meta_webhook.py

HOW TO CHECK Send four photos quickly and tap Pack on each: the fourth reply says 'You already have 3 orders being prepared, nothing was charged'.

3. Customer experience and edges

- ☐ **3.1 The processing message uses the measured time of recent orders instead of a fixed '20-30 seconds'**

FILES backend/app/services/eta_service.py

HOW TO CHECK After three delivered orders the message reads 'Usually ready in about N minutes'. Until then it keeps the original wording.
- ☐ **3.2 Cheaper request handling, size-checked uploads, API docs off in production, admin-only status endpoint**

FILES backend/app/middleware/, backend/app/utils/upload_limits.py, backend/app/main.py

HOW TO CHECK Upload a file bigger than 10 MB: refused with a clear message. With ENVIRONMENT=production /docs answers 404.

4. Not built (owner decisions)

- ☐ **4.1 Group a burst of photos into one confirmation (UX-1)**

BLOCKED BY How a batch is priced and confirmed: see docs/QUESTIONS_FOR_MORNING.md

- ☐
- 4.2 Move customer photos to object storage and sign image links (DEP-3, SEC-7)**

BLOCKED BY The owner's choice of storage provider and an account.
-

5. Deployment files

- ☐
- 5.1 Dockerfile, compose file and HTTPS proxy config with graceful shutdown**

FILES backend/Dockerfile, docker/docker-compose.yml, docker/Caddyfile

HOW TO CHECK Written but never built: this machine has no Docker. The first docker compose up on a real server is the real test.
-

Key decisions and why

D1 Use a database outbox (not Redis and Celery) to make paid orders survive restarts.

WHY THIS CHOICE

The database already exists and already holds the order state; the outbox is a table plus a sweep. It removes the 'lost on restart' risk now without the owner choosing and paying for a new service.

OTHER OPTIONS CONSIDERED (AND WHY NOT)

- **Celery with Redis** — not chosen because: Needs a hosting and Redis decision the owner has not made; kept as a later step if one server cannot cope.
- **Keep in-memory background tasks and rely on the recovery sweep** — not chosen because: Orders are refunded after 10 minutes instead of being served.

Relates to task(s): 1.1

D2 Run the order directly as before AND record it in the outbox first; the sweep only rescues orders nobody started after 90 seconds.

WHY THIS CHOICE

Customers see no change in speed, the server's shutdown still waits for orders in progress, and a crash can no longer lose an order.

OTHER OPTIONS CONSIDERED (AND WHY NOT)

- **Let only the sweep run orders** — not chosen because: Adds up to 20 seconds of delay to every order.

Relates to task(s): 1.1

D3 A per-provider circuit breaker with one probe after a cool-down, counting one failure per order (not per retry), and never counting refused prompts.

WHY THIS CHOICE

During an outage this makes orders fail fast and refund instead of waiting for several timeouts, while a blocked prompt (the customer's photo) is not the provider's fault.

OTHER OPTIONS CONSIDERED (AND WHY NOT)

- **Count every failed attempt** — not chosen because: Two ordinary rate-limited orders would open the circuit.
- **No breaker, rely on timeouts** — not chosen because: Every order waits through the full timeout during an outage.

Relates to task(s): 2.1

D4 Do not build photo-burst grouping (UX-1), object storage (DEP-3) or the full Celery queue yet.

WHY THIS CHOICE

Each needs a decision only the owner can make (how a batch is priced, which storage provider, whether to pay for Redis). They are recorded in docs/QUESTIONS_FOR_MORNING.md.

OTHER OPTIONS CONSIDERED (AND WHY NOT)

- **Choose for the owner** — not chosen because: Changes pricing and costs without their say.

Errors faced

E1 Reviewer: circuit breaker could stay open forever if the probe call was cancelled

Cause: The probe slot was only freed by a success, a failure or a neutral report; a cancelled probe reported nothing.

Fix: Every call chain now always reports an outcome (even on cancel or exception), and a probe that never reports expires after the cool-down.

Task: 2.1

E2 Reviewer: a re-delivered payment could send its invoice twice

Cause: 'Could not queue' and 'already queued' both looked like failure, so the code fell back to sending directly.

Fix: The outbox now says queued, duplicate or unavailable; a duplicate does nothing.

Task: 1.2

E3 TimeoutError: QueuePool limit of size 5 overflow 10 reached (load scenario k)

Cause: The new queue-depth lookup used a second database connection while the request still held its own.

Fix: End the request's transaction before the lookup.

Task: 3.1

E4 Reviewer: in Docker every internet visitor shared one rate-limit bucket

Cause: uvicorn did not trust the proxy's forwarded address, so all traffic looked like it came from the proxy.

Fix: Start uvicorn with --proxy-headers and trust the private proxy network.

Task: 5.1